

CareConnect: Smart Hospital Matching System

A PROJECT REPORT

Submitted by

Abraham Tamene – UUR00322/16

Daniel Hailay – UUR02022/16

Daniel Lemma – UUR00362/16

Feven Alemayehu – UUR00037/17

Fiker Belay – UUR00180/17

In partial fulfillment for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Under the guidance of

Zebiba Yusuf Sufyan



DEPARTMENT OF COMPUTER SCIENCE

UNITY UNIVERSITY, GERJI CAMPUS

ADDIS ABABA

MONTH AND YEAR

April 2026

ABSTRACT

Our project, CareConnect, proposes a smart hospital matching system designed to help patients find healthcare facilities that best fit their needs. Choosing the right hospital can be difficult, especially when factors such as illness type, budget, location, and personal care preferences all play a role. Many existing systems lack personalized filtering, which often leads to delays, added stress, and less informed decisions for patients and their families.

CareConnect addresses this gap by offering a more tailored approach to hospital selection. The system allows users to search for healthcare facilities based on their specific conditions and preferences, providing ranked results to support better decision making. Its core features include patient and hospital registration, illness based search, appointment requests, and a feedback system.

The application is built using object-oriented design principles, with key components such as patients, hospitals, and a matching engine modeled accordingly. In addition to functional requirements, the system also emphasizes performance, data privacy, and ease of use.

This report includes UML representations such as use case diagrams and CRC cards. Overall, the project demonstrates how object-oriented analysis and design can be applied to improve healthcare accessibility and decision making.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our advisor, Zebiba Yusuf Sufyan, for her guidance, support, and valuable feedback throughout this project. Her encouragement helped us stay on track and complete this work.

We also thank Unity University, Department of Computer Science, for providing the resources and environment to carry out this research. We are grateful to our families and friends for their patience and encouragement during the development of this project.

Finally, we would like to thank each other for the work we did on this project and for having one another's back throughout the whole process.

Contents

ABSTRACT	i
ACKNOWLEDGEMENT	ii
LIST OF TABLES.....	v
LIST OF FIGURES.....	vi
LIST OF SYMBOLS, ABBREVIATIONS, ACRONYMS.....	viii
CHAPTER ONE: INTRODUCTION.....	1
1.1 Introduction	1
1.2 Background of the Project.....	1
1.3 Motivation and Justification.....	2
1.4 Problem Statement	2
1.5 Objective of the Project	2
1.5.1 General Objective	2
1.5.2 Specific Objectives	3
1.6 Scope of the Project	3
1.7 Limitations of the Project	3
1.8 Significance of the Project	4
1.9 Methodology.....	4
1.9.1 Data Collection Methodology.....	4
1.10 System Requirements.....	4
1.10.1. Hardware Requirement	4
1.10.2. Software Requirement	5
1.10.3. Programming Language	5
CHAPTER TWO: REQUIREMENT GATHERING	5
2.1 Existing System Description	5
2.1.1 Description of the Major Activities in the Existing System	5
2.1.2 Functional Work Flow of the Existing System.....	6
2.2 Problem Definition.....	6
2.3 Broad Alternative Solutions	7

2.4 Functional Requirements of the Proposed System	7
2.5 Nonfunctional Requirements	8
2.6 Business Rule Documentation	8
2.7 Use Case Modeling	9
2.7.1 Essential Use Case Modeling.....	9
2.8 Class Responsibility Collaboration Modeling	10
CHAPTER THREE: SYSTEM ANALYSIS	11
3.1 System Use Case Modeling	11
3.2 System Use Case Description.....	14
3.3 Analysis Sequence Diagrams	22
3.4 Conceptual Modeling: Class Diagram.....	25
3.5 Analysis Class Documentation.....	27
3.6 Activity Diagrams	28
CHAPTER FOUR: SYSTEM DESIGN.....	33
4.1 Design Class Modeling	33
4.1.1 Design Class Diagrams.....	33
4.1.2 Design Class Documentation.....	35
4.2 Collaboration Diagrams	36
4.3 Component Diagrams	38
4.4 Deployment Diagram	40
4.5 Database Design	41
4.5.1 Persistence Modeling.....	41
4.5.2 Normalization.....	43
4.6 User Interface Design.....	44
REFERENCE	47

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Functional Requirements of CareConnect	7
Table 2.2	Non-Functional Requirements of CareConnect	8
Table 3.1	use case Register	14
Table 3.2	use case Login	15
Table 3.3	use case Search Hospitals	16
Table 3.4	use case Request Appointment	17
Table 3.5	use case Confirm/Decline Appointment	18
Table 3.6	use case Submit Feedback	19
Table 3.7	use case Manage Hospital Listings	20
Table 3.8	use case Generate Reports	21
Table 3.9	Analysis Class Documentation	27 - 28
Table 4.1	Design Class Documentation	35-36

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 2.1	Workflow diagram of existing system	6
Figure 2.2	Use Case Diagram for CareConnect	9
Figure 2.3	CRC Cards for Key Classes	11
Figure 3.1	Updated Use Case Diagram (with include/extend)	13
Figure 3.2a	Sequence Diagram – Search Hospitals	23
Figure 3.2b	Sequence Diagram – Request Appointment	23
Figure 3.2c	Sequence Diagram – Confirm/Decline Appointment	24
Figure 3.2d	Sequence Diagram – Register	24
Figure 3.2e	Sequence Diagram – Submit Feedback	25
Figure 3.3	Conceptual Class Diagram for CareConnect	26
Figure 3.4	Activity Diagram – Search Hospitals	28
Figure 3.5	Activity Diagram – Register	29

Figure 3.6	Activity Diagram – Request Appointment	30
Figure 3.7	Activity Diagram – Confirm/Decline Appointment	31
Figure 3.8	Activity Diagram – Submit Feedback	32
Figure 4.1	Design Class Diagram for CareConnect	34
Figure 4.2	Collaboration Diagram – Search Hospitals	38
Figure 4.3	Component Diagram for CareConnect	40
Figure 4.4	Deployment Diagram for CareConnect	41
Figure 4.5	Persistence (ER) Diagram for CareConnect	43
Figure 4.6	User Interface Flow Diagram for CareConnect	46

LIST OF SYMBOLS, ABBREVIATIONS, ACRONYMS

Abbreviation / Symbol	Meaning
OOP	Object-Oriented Programming
UML	Unified Modeling Language
CRC	Class-Responsibility-Collaboration
UI	User Interface
FR	Functional Requirement
NFR	Non-Functional Requirement
BR	Business Rule
UC	Use Case

CHAPTER ONE: INTRODUCTION

1.1 Introduction

Healthcare decision making is often stressful for patients and their families, especially when they need to find a hospital that matches their specific illness, budget, and location. Many existing solutions only offer basic directory listings without intelligent filtering. This project, CareConnect, aims to solve that problem by providing a smart hospital matching system built using object-oriented principles. This chapter presents the background, problem statement, objectives, scope, methodology, and system requirements for CareConnect.

1.2 Background of the Project

In many countries, patients struggle to find appropriate hospitals because information is scattered across different websites, phone directories, or word of mouth. Even when a hospital is found, it may not accept the patient's budget or offer the required medical specialty. CareConnect was conceived as a response to this fragmentation. By applying object-oriented analysis and design, the system models real world entities (patients, hospitals, appointments) as software objects, making the matching process more natural and adaptable. The background of this project is rooted in the need for a unified, intelligent platform that reduces the time and stress involved in finding the right healthcare facility.

1.3 Motivation and Justification

The motivation for CareConnect comes from real world difficulties patients face when trying to find a suitable hospital. Families often waste time calling multiple hospitals or traveling long distances only to discover that services are too expensive or the required medical specialty is not available. By creating a smart matching system, we aim to reduce this burden. The justification for using object-oriented design is that it allows us to model patients, hospitals, and matching logic as reusable software components, making the system easier to maintain and extend in the future.

1.4 Problem Statement

Many patients face difficulty in identifying hospitals that meet their specific medical, financial, and personal needs. Existing systems do not provide personalized filtering based on illness type, budget, location, and required healthcare services. This often results in delays, poor decision making, and increased stress for patients and their families.

1.5 Objective of the Project

1.5.1 General Objective

To develop a smart hospital matching system that helps users find the most suitable healthcare facility based on their illness, budget, location, and care preferences.

1.5.2 Specific Objectives

- To design an object-oriented model for patient and hospital entities.
- To implement a matching algorithm that ranks hospitals by illness specialty, budget affinity, and geographic proximity.
- To provide a user friendly interface for search and appointment requests.
- To ensure data privacy and system reliability.

1.6 Scope of the Project

The system will cover the following features:

- Patient registration and login.
- Hospital registration and service management.
- Search with filters: illness, budget range, location.
- Ranked list of matching hospitals.
- Appointment request and confirmation.
- Feedback submission.

The following are out of scope for this phase: real time bed availability, payment processing, telemedicine integration, and mobile application development. These may be considered in future versions.

1.7 Limitations of the Project

The following limitations are acknowledged for the current phase: real time bed availability is not included, telemedicine integration is out of scope, and the mobile application is not yet developed.

1.8 Significance of the Project

The significance of this project lies in reducing patient stress, saving time, and providing transparent hospital comparisons, as outlined in the problem statement.

1.9 Methodology

The project follows an iterative development approach, beginning with the Initiate phase (problem definition and requirements). This report covers Chapters 1 and 2 as part of the Initiate phase.

1.9.1 Data Collection Methodology

- Surveys
- Interviews
- existing hospital data

1.10 System Requirements

1.10.1. Hardware Requirement

- Server: 8 GB RAM, 100 GB storage, Intel i5 or equivalent
- Client: Any computer with a modern browser (Chrome, Firefox)

1.10.2. Software Requirement

- Operating System: Windows 10 / Linux / macOS
- Database: MySQL (or PostgreSQL)
- Web Server: Apache / Nginx
- UML Tool: Lucidchart or Microsoft Visio (for diagrams)

1.10.3. Programming Language

- Backend: Python with Django (or Java with Spring Boot)
- Frontend: HTML, CSS, JavaScript
- Query Language: SQL

CHAPTER TWO: REQUIREMENT GATHERING

2.1 Existing System Description

Current hospital search systems are mostly static directories. They do not rank hospitals by illness type or patient budget.

2.1.1 Description of the Major Activities in the Existing System

- Manual browsing of hospital lists
- Calling hospitals individually to check services
- No comparison of costs or specialties

2.1.2 Functional Work Flow of the Existing System

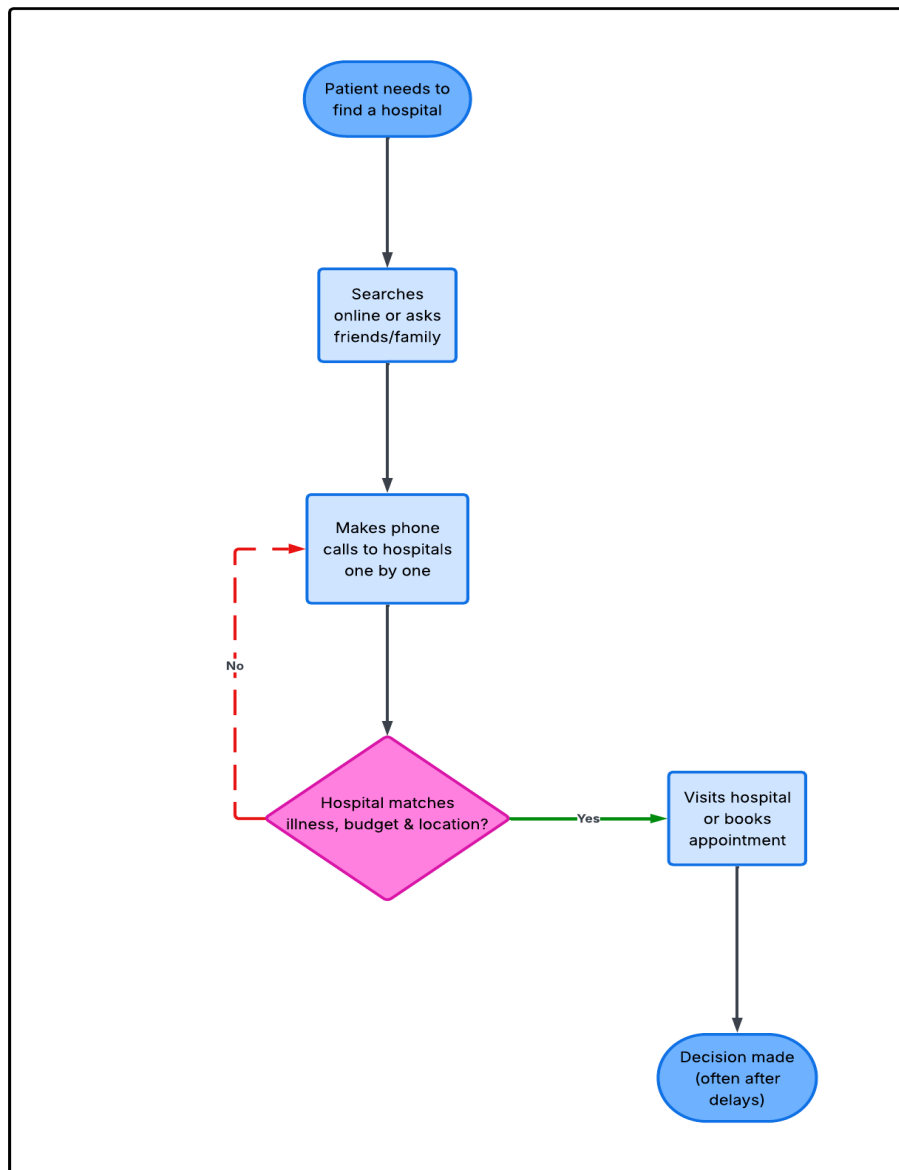


Figure 2.1: Workflow diagram of existing system

2.2 Problem Definition

Patients waste time and money because no system matches their illness to the right hospital specialty, budget, and location. There is also no unified way to book appointments after filtering.

2.3 Broad Alternative Solutions

- Manual referral by doctors (trustworthy but not scalable)
- General search engines (broad but no medical filtering)
- Dedicated matching system – CareConnect (chosen)

2.4 Functional Requirements of the Proposed System

ID	Requirement,	Description
FR1	Patient registration	Patient creates account with personal and medical info (encrypted)
FR2	Hospital registration	Admin adds hospital license, address, services, price ranges
FR3	Search hospitals	Patient enters illness, budget, location → ranked list
FR4	Appointment request	Patient sends request to a hospital
FR5	Confirm/decline appointment	Hospital admin responds
FR6	Submit feedback	Patient reviews experience

Table 2.1: Functional Requirements of CareConnect

2.5 Nonfunctional Requirements

ID	Requirement,	Target
NFR1	Response time	Match results within 3 seconds
NFR2	Data privacy	Encryption of medical and financial data
NFR3	Availability	99.5% uptime
NFR4	Usability	Complete search in ≤ 3 clicks
NFR5	Maintainability	OOP design allows new filters without breaking core

Table 2.2: Non-Functional Requirements of CareConnect

2.6 Business Rule Documentation

- BR-CC-001 (Matching logic constraints): Illness severity, budget alignment, and geographic proximity are weighted.
- BR-CC-002 (Illness-to-Specialty Mapping): Each illness maps to one or more hospital specialties (e.g., Cardiology for heart disease).
- BR-CC-003 (Geographic Proximity Weighting): Hospitals within 10 km get a 30% ranking boost; within 50 km get 10%.
- Data privacy rule: Patient medical data accessible only through `getIllnessSummary()` method (encapsulation). Hospital financial data visible only to authorized admins.

2.7 Use Case Modeling

2.7.1 Essential Use Case Modeling

Primary actors: Patient, Hospital Administrator, System Administrator

Secondary actor: Notification Service (external)

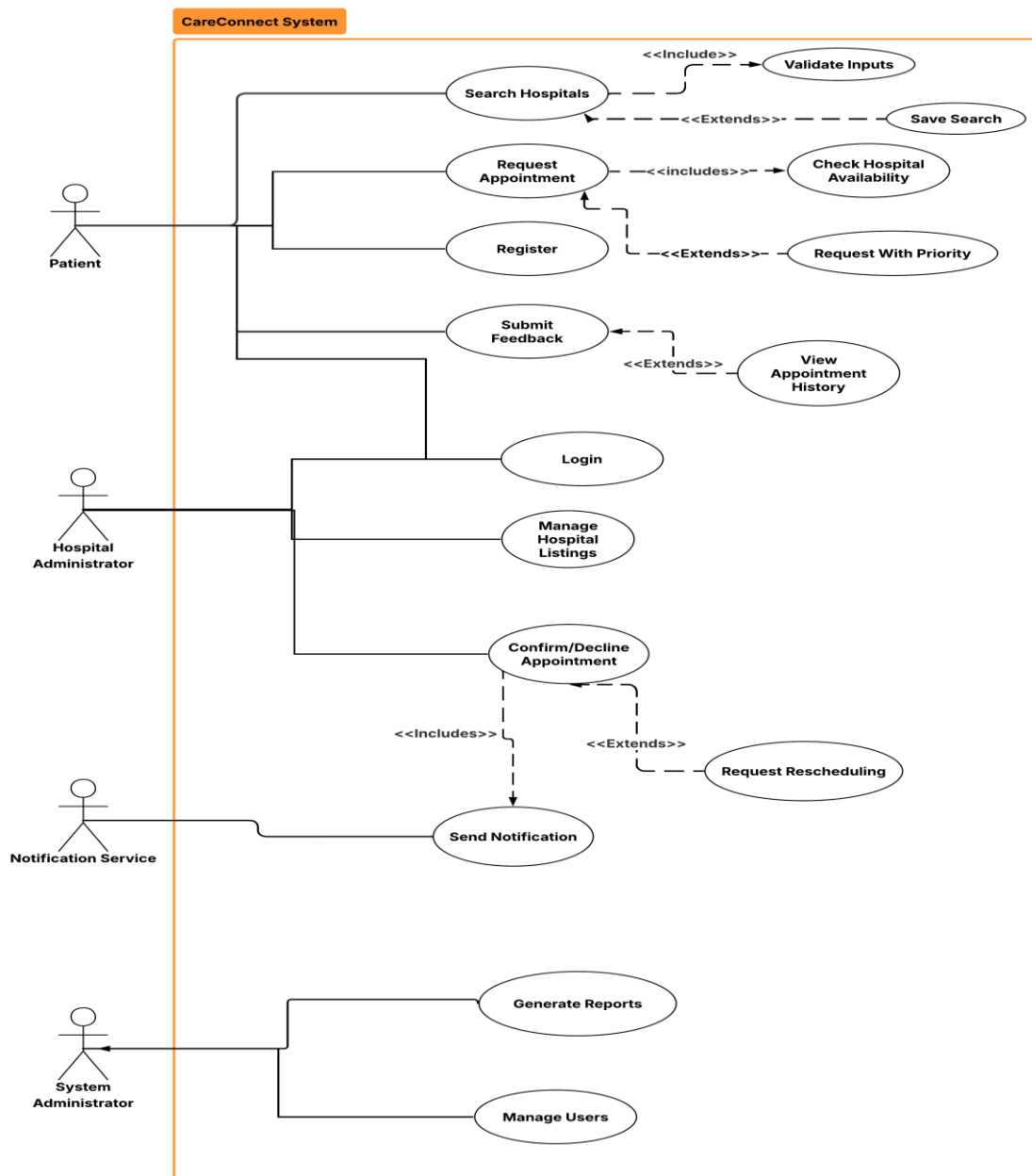


Figure 2.2: Use Case Diagram for CareConnect

2.7.1.1 Essential Use Case Documentation

Use Case 1: Search for Hospitals

- Actor: Patient
- Precondition: Patient logged in
- Main flow: Enter illness, budget, location → system validates → invokes matching algorithm → displays ranked list.
- Post condition: Patient sees matching hospitals.

Use Case 2: Register Hospital

- Actor: Hospital Admin
- Main flow: Admin provides license, address, specialties, and price list → system stores data.

2.8 Class Responsibility Collaboration Modeling

Candidate classes: Patient, Hospital, Appointment, Review, MatchingEngine, User, HospitalAdmin, NotificationService.

Refined classes and CRC cards are shown as followed:

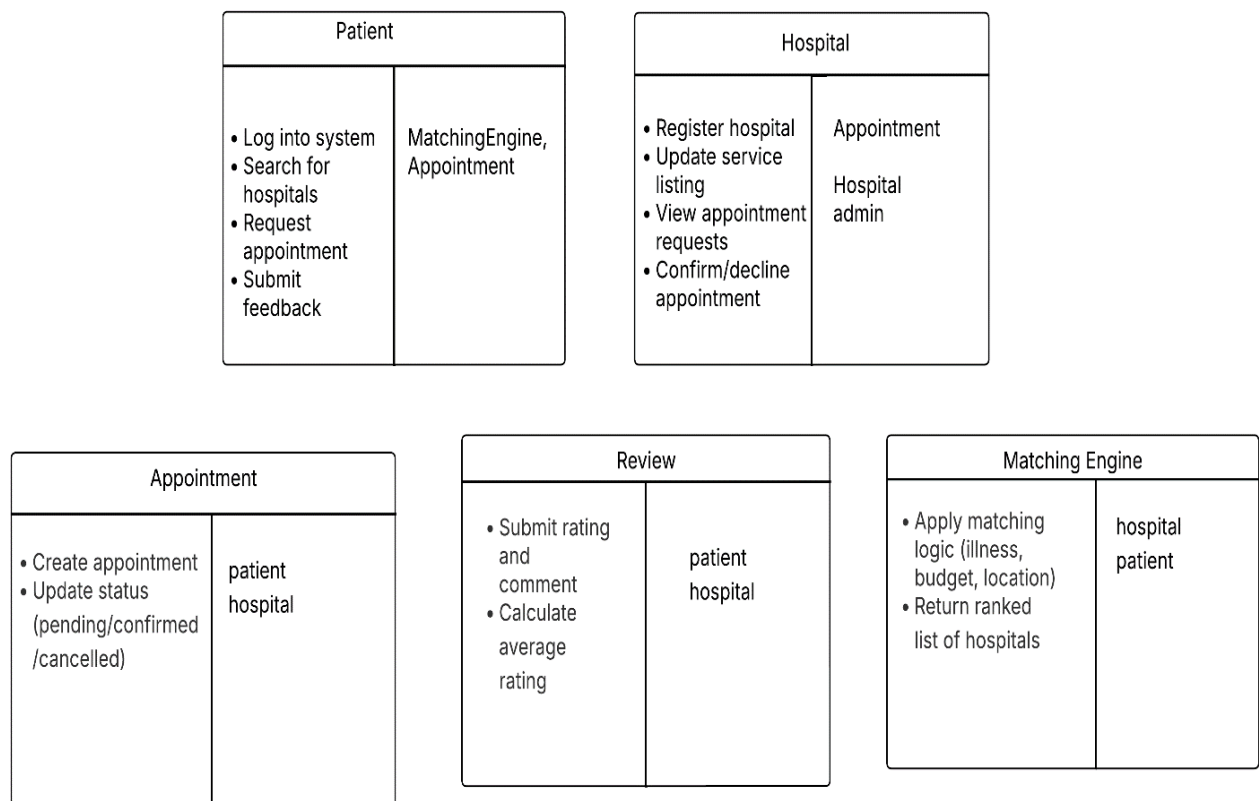


Figure 2.3: CRC Cards for Key Classes.

CHAPTER THREE: SYSTEM ANALYSIS

3.1 System Use Case Modeling

The system use case model refines the essential use case model from Chapter 2 by adding include and extend relationships. The essential model focused only on core user goals, deliberately omitting technical or cross-cutting behaviors. In Chapter 3, during system analysis, we introduce these relationships for two reasons: (1) to identify reusable, mandatory sub-functions (using <<include>>), and (2) to capture optional or exceptional user actions (using <<extend>>). This

expansion is normal when moving from essential to system-level use cases it adds implementation relevant details without altering the original user goals.

Actors remain the same: Patient, Hospital Administrator, System Administrator, and Notification Service (external).

The main use cases are:

- Register
- Login
- Search Hospitals
- Request Appointment
- Confirm/Decline Appointment
- Submit Feedback
- Manage Hospital Listings
- Generate Reports (for System Administrator)

To improve clarity and reuse, the following include (<<include>>) and extend (<<extend>>) relationships are defined:

Include relationships (<<include>>)

- Login includes Validate Credentials
- Search Hospitals includes Validate Inputs
- Request Appointment includes Check Hospital Availability
- Confirm/Decline Appointment includes Send Notification
- Submit Feedback includes Validate Feedback Content
- Register includes Encrypt Sensitive Data

Extend relationships (<<extend>>)

- Save Search extends Search Hospitals (extension point: after search results are displayed)

- Reschedule Appointment extends Confirm/Decline Appointment (when appointment is declined)
- Request Priority extends Request Appointment (when emergency box is checked)
- Forgot Password extends Login (when user clicks “forgot password”)
- Export Results extends Search Hospitals (optional download)

The updated use case diagram is presented in Figure 3.1.

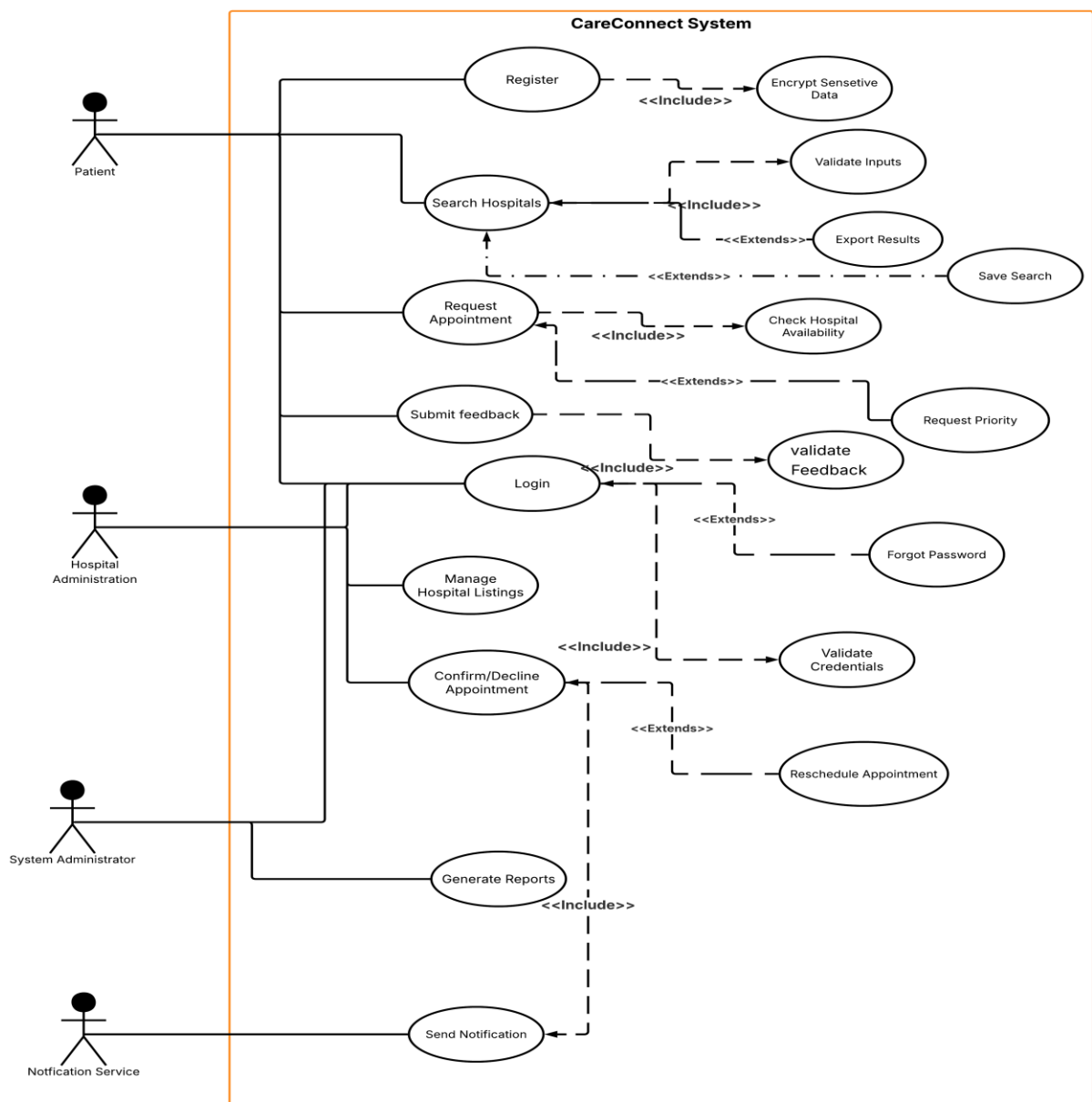


Figure 3.1 – Updated Use Case Diagram (with include/extend)

3.2 System Use Case Description

Field	Content
Use Case ID	UC-01
Use Case Name	Register
Actor	Patient or Hospital Administrator
Description	Allows a new user to create an account in the system.
Precondition	User is not logged in.
Post condition	A new account is created and user is redirected to login.
Flow of Events (Basic)	1. User selects “Register”. 2. System displays registration form. 3. User enters required information (name, email, password, and role). 4. System validates input. 5. System encrypts sensitive data (e.g., password). 6. System stores the account. 7. System confirms registration and asks user to log in.
Alternative Flows	4a. If input is invalid (e.g., missing fields, invalid email), system prompts for correction. 5a. If email already exists, system notifies user and suggests login.

Table 3.1 – use case Register

Field	Content
Use Case ID	UC-02
Use Case Name	Login
Actor	Patient, Hospital Administrator, or System Administrator
Description	Allows registered users to access the system.
Precondition	User has a registered account.
Post condition	User is logged in and redirected to their dashboard.
Flow of Events (Basic)	1. User enters email and password. 2. System validates credentials. 3. System retrieves the user's role. 4. System grants access and redirects to appropriate dashboard.
Alternative Flows	2a. If credentials are invalid, system denies access and shows error message. 4a. If user forgets password, they can use "Forgot Password" extension.

Table 3.2 – use case Login

Field	Content
Use Case ID	UC-03
Use Case Name	Search Hospitals
Actor	Patient
Description	Allows a patient to find hospitals matching illness, budget, and location.
Precondition	Patient is logged in.
Post condition	A ranked list of matching hospitals is displayed.
Flow of Events (Basic)	1. Patient enters illness, budget range, and location. 2. System validates input. 3. System invokes matching algorithm. 4. System retrieves and ranks hospitals. 5. System displays ranked list. 6. Patient may select a hospital for details.
Alternative Flows	2a. If input is invalid, system prompts for correction. 5a. If no hospitals match, system shows “No results found” and suggests changing criteria. 6a. Patient may save search (extends to UC-07).

Table 3.3 – use case Search Hospitals

Field	Content
Use Case ID	UC-04
Use Case Name	Request Appointment
Actor	Patient
Description	Allows a patient to request an appointment with a selected hospital.
Precondition	Patient has performed a search and selected a hospital.
Post condition	Appointment request is created and notification is sent to hospital.
Flow of Events (Basic)	1. Patient selects “Request Appointment”. 2. System checks hospital availability. 3. System displays appointment form (date, reason). 4. Patient fills and submits form. 5. System validates information. 6. System creates request with status “pending”. 7. System notifies hospital admin.
Alternative Flows	2a. If hospital is unavailable, system informs patient and suggests alternatives. 5a. If input invalid, system prompts correction. 7a. If patient checks “emergency”, system flags request as priority.

Table 3.4 – use case Request Appointment

Field	Content
Use Case ID	UC-05
Use Case Name	Confirm/Decline Appointment
Actor	Hospital Administrator
Description	Allows hospital admin to accept or reject an appointment request.
Precondition	Appointment request exists with status “pending”.
Post condition	Request status is updated (confirmed or declined), and patient is notified.
Flow of Events (Basic)	1. Admin views list of pending requests. 2. Admin selects a request. 3. System displays request details. 4. Admin chooses “Confirm” or “Decline”. 5. System updates status. 6. System sends notification to patient.
Alternative Flows	4a. If decline, system may prompt for reason (optional). 6a. If declined, patient may request reschedule (extends to UC-08).

Table 3.5 – use case Confirm/Decline Appointment

Field	Content
Use Case ID	UC-06
Use Case Name	Submit Feedback
Actor	Patient
Description	Allows a patient to rate and comment on a hospital after an appointment.
Precondition	Appointment status is “confirmed”.
Post condition	Feedback is stored and hospital’s average rating is updated.
Flow of Events (Basic)	1. Patient selects “Submit Feedback”. 2. System displays rating and comment form. 3. Patient enters rating (1–5) and comment. 4. System validates content. 5. System stores feedback. 6. System updates hospital’s average rating.
Alternative Flows	4a. If comment contains inappropriate content, system rejects and asks for correction.

Table 3.6 – use case Submit Feedback

Field	Content
Use Case ID	UC-07
Use Case Name	Manage Hospital Listings
Actor	Hospital Administrator
Description	Allows hospital admin to add, update, or remove hospital services, pricing, and contact details.
Precondition	Admin is logged in and registered as a hospital.
Post condition	Hospital information is updated in the system.
Flow of Events (Basic)	1. Admin selects “Manage Listings”. 2. System displays current hospital information. 3. Admin modifies fields (services, price list, address, etc.). 4. System validates changes. 5. System updates the database. 6. System confirms success.
Alternative Flows	4a. If invalid data (e.g., negative price), system rejects and prompts correction.

Table 3.7 – use case Manage Hospital Listings

Field	Content
Use Case ID	UC-08
Use Case Name	Generate Reports
Actor	System Administrator
Description	Allows system admin to generate usage or performance reports.
Precondition	Admin is logged in.
Post condition	A report (e.g., number of searches, appointment trends) is generated and available for download.
Flow of Events (Basic)	1. Admin selects “Generate Reports”. 2. System displays report types (daily, weekly, by hospital, etc.). 3. Admin chooses type and date range. 4. System queries data. 5. System generates report (PDF or CSV). 6. System offers download.
Alternative Flows	4a. If no data in range, system displays “No data available”.

Table 3.8 – use case Generate Reports

3.3 Analysis Sequence Diagrams

Sequence diagrams model the flow of logic within a system, showing how actors and objects interact over time. For CareConnect, we present sequence diagrams for the core system use cases: Search Hospitals, Request Appointment, Confirm/Decline Appointment, and Register. Each diagram illustrates the interaction between actors (Patient, Hospital Administrator, or User), user interface components, controllers, services, and databases.

- Figure 3.2a shows the flow for Search Hospitals, including validation, matching algorithm invocation, and result display.
- Figure 3.2b shows the flow for Request Appointment, including availability check and notification.
- Figure 3.2c shows the flow for Confirm/Decline Appointment, including status update and patient notification.
- Figure 3.2d shows the flow for Register, including data encryption and account storage.
- Figure 3.2e shows the flow for Submit Feedback, including content validation, storing feedback, and updating hospital ratings.

These diagrams together validate the dynamic behavior of the system and serve as a basis for class modeling and implementation.

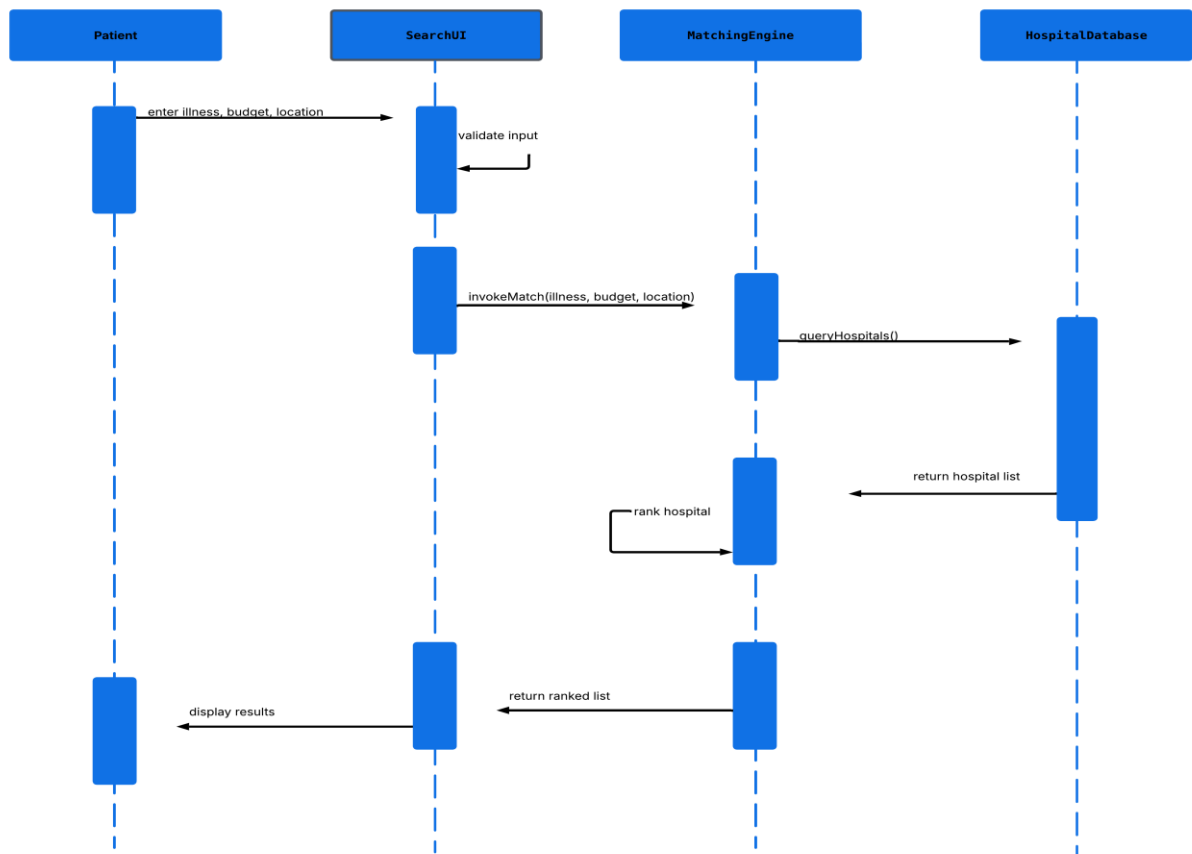


Figure 3.2a: Sequence Diagram – Search Hospitals

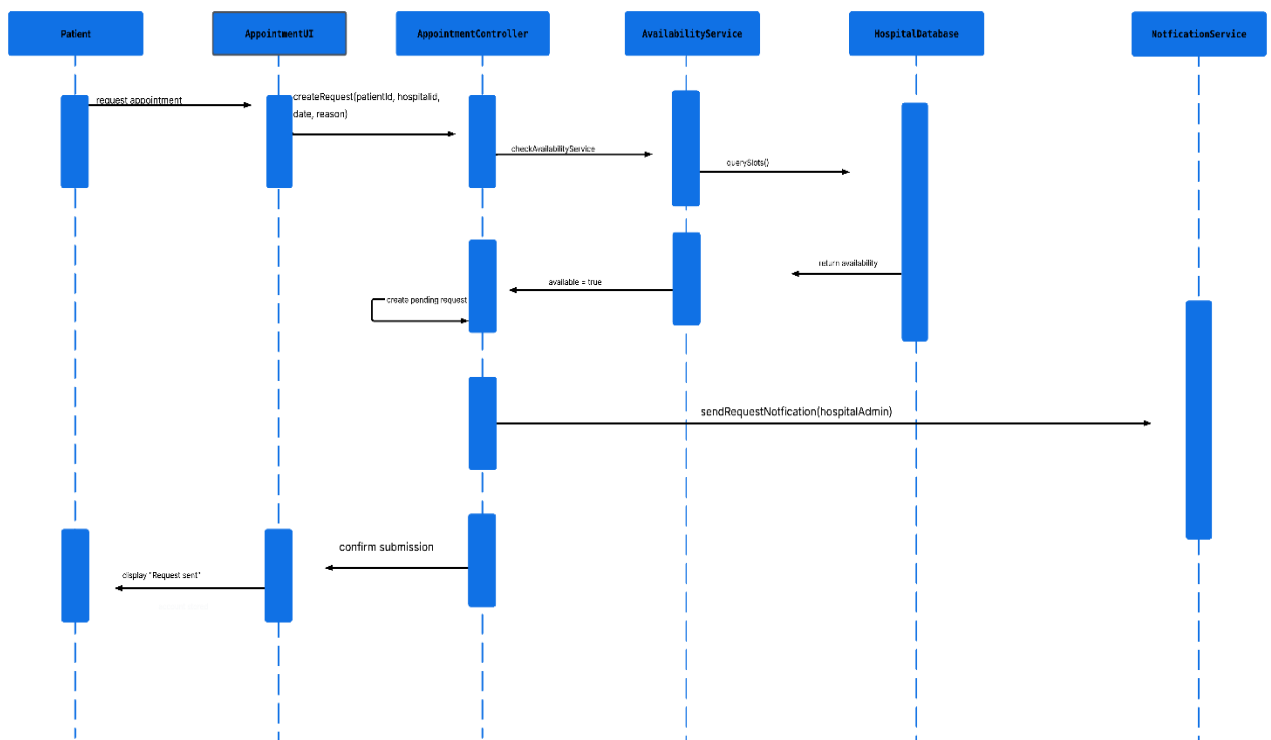


Figure 3.2b: Sequence Diagram – Request Appointment

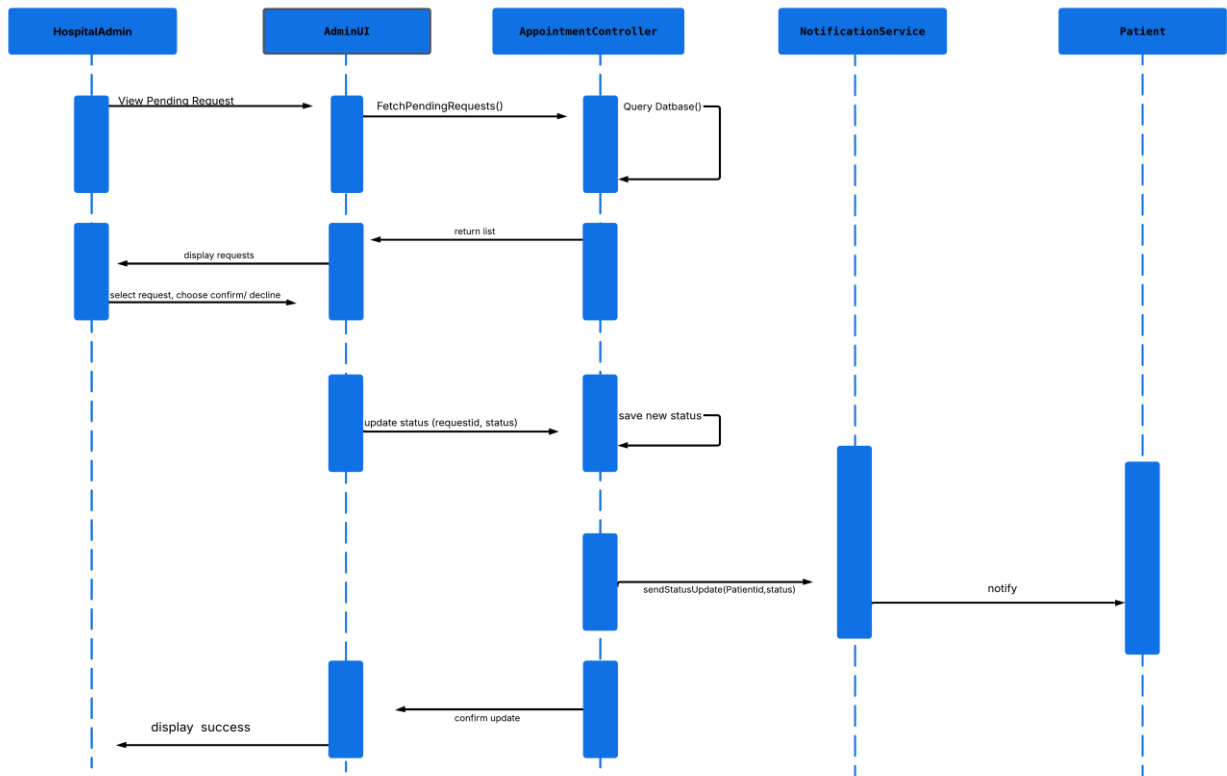


Figure 3.2c: Sequence Diagram – Confirm/Denial Appointment

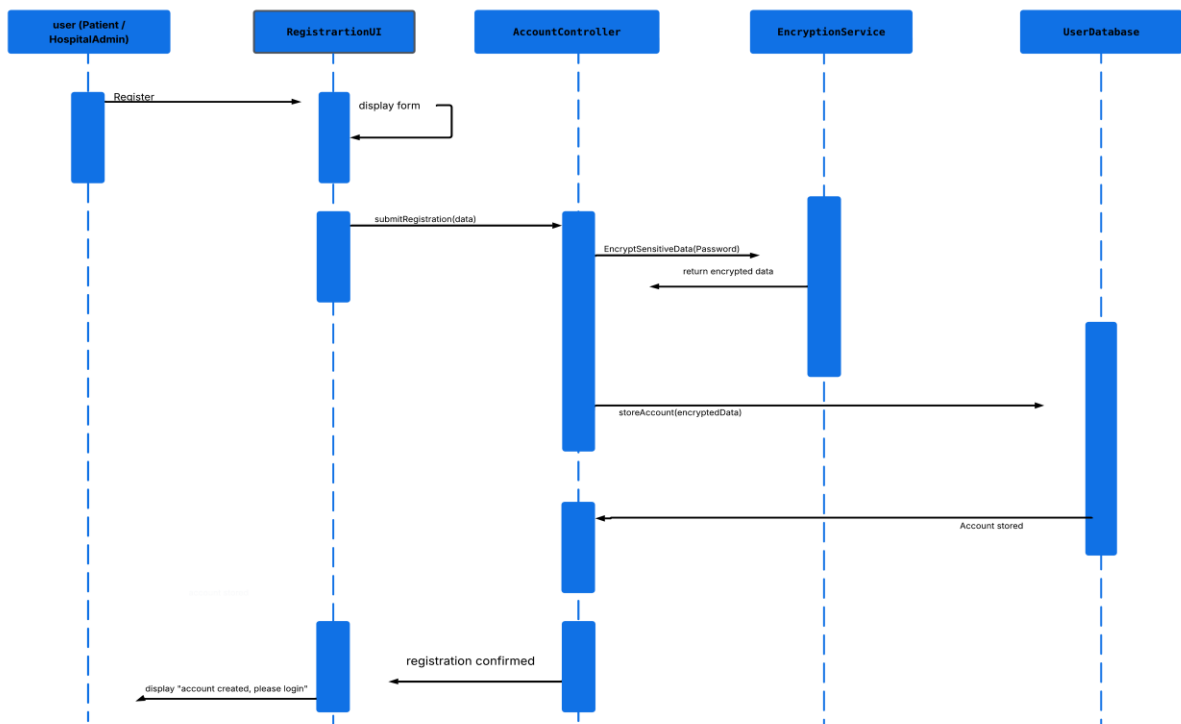


Figure 3.2d: Sequence Diagram – Register

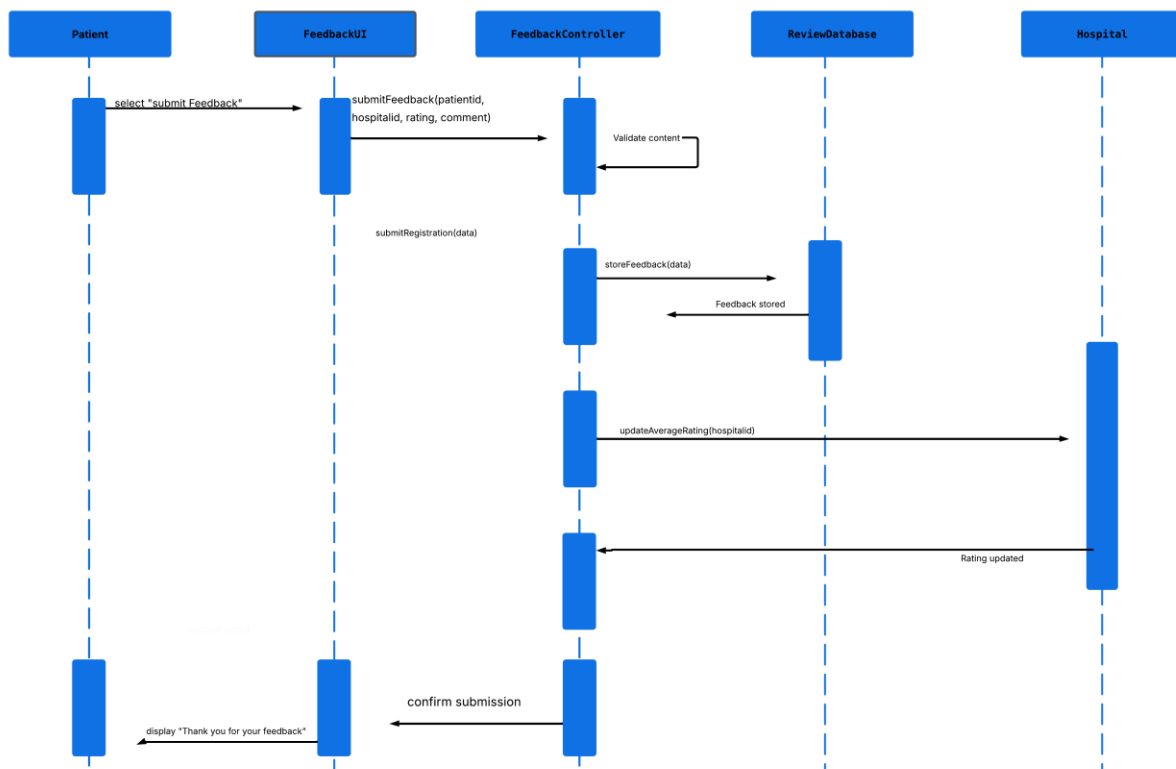


Figure 3.2e: Sequence Diagram – Submit Feedback

3.4 Conceptual Modeling: Class Diagram

The conceptual class diagram models the static structure of the CareConnect system. It is derived from the CRC cards (Figure 2.3) and refined with attributes, methods, and associations. The main classes are: Patient, Hospital, Appointment, Review, MatchingEngine, User (abstract), HospitalAdmin, SystemAdmin, and NotificationService.

Key associations and multiplicities:

- A Patient can make zero or many Appointments (1 — 0..*).
- A Hospital can have zero or many Appointments (1 — 0..*).
- A Hospital can receive zero or many Reviews (1 — 0..*).

- A Patient can write zero or many Reviews (1 — 0..*).
- The MatchingEngine is invoked by the Search Hospitals use case but does not maintain persistent associations.

Inheritance: HospitalAdmin and SystemAdmin inherit from User. Patient also inherits from User (if email/password are shared).

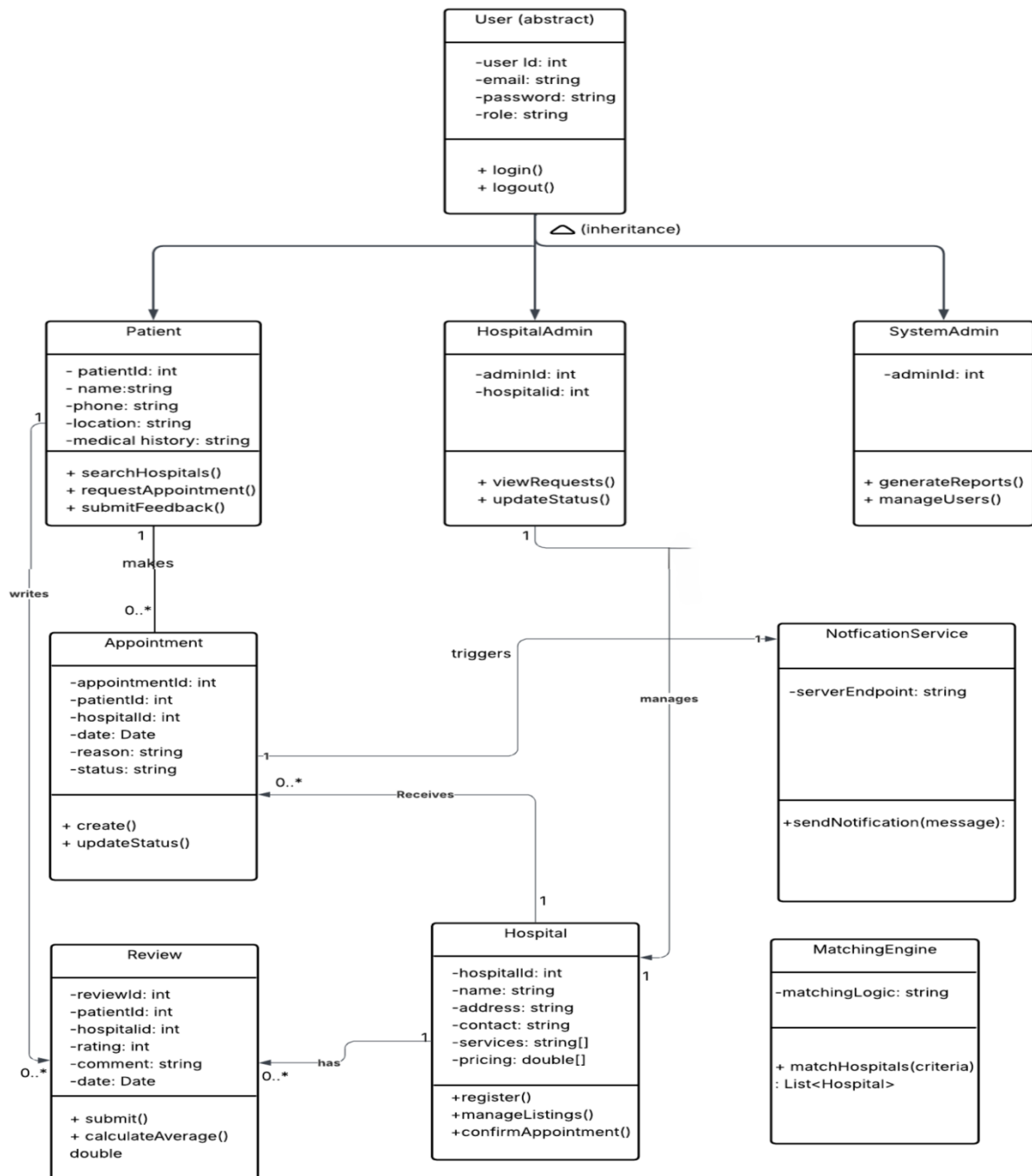


Figure 3.3: Conceptual Class Diagram for CareConnect

3.5 Analysis Class Documentation

The following table describes the key analysis classes identified in the conceptual class diagram (Figure 3.3). Classes are categorized as entity (persistent data), control (business logic), or boundary (user interface).

Class	Type	Description
Patient	Entity	Represents a patient user, stores personal and medical data.
Hospital	Entity	Represents a healthcare facility, stores services, pricing, contact.
Appointment	Entity	Stores appointment requests and their status.
Review	Entity	Stores patient ratings and comments for a hospital.
User	Entity	Abstract class, holds common login credentials and role.
HospitalAdmin	Entity	Extends User, manages hospital listings and appointments.
SystemAdmin	Entity	Extends User, generates reports and manages system.
MatchingEngine	Control	Implements matching algorithm for hospital search.
NotificationService	Control	Sends notifications to patients and admins.
SearchUI	Boundary	User interface for search and results display.

AppointmentUI	Boundary	User interface for requesting appointments.
AdminUI	Boundary	User interface for hospital/admin dashboards.

Table 3.9 – Analysis Class Documentation

3.6 Activity Diagrams

An activity diagram models the dynamic flow of control from one activity to another. For CareConnect, we present activity diagrams for the Search Hospitals, Register, Request Appointment, Confirm/Decline Appointment and Submit Feedback use cases.



Figure 3.4: Activity Diagram – Search Hospitals

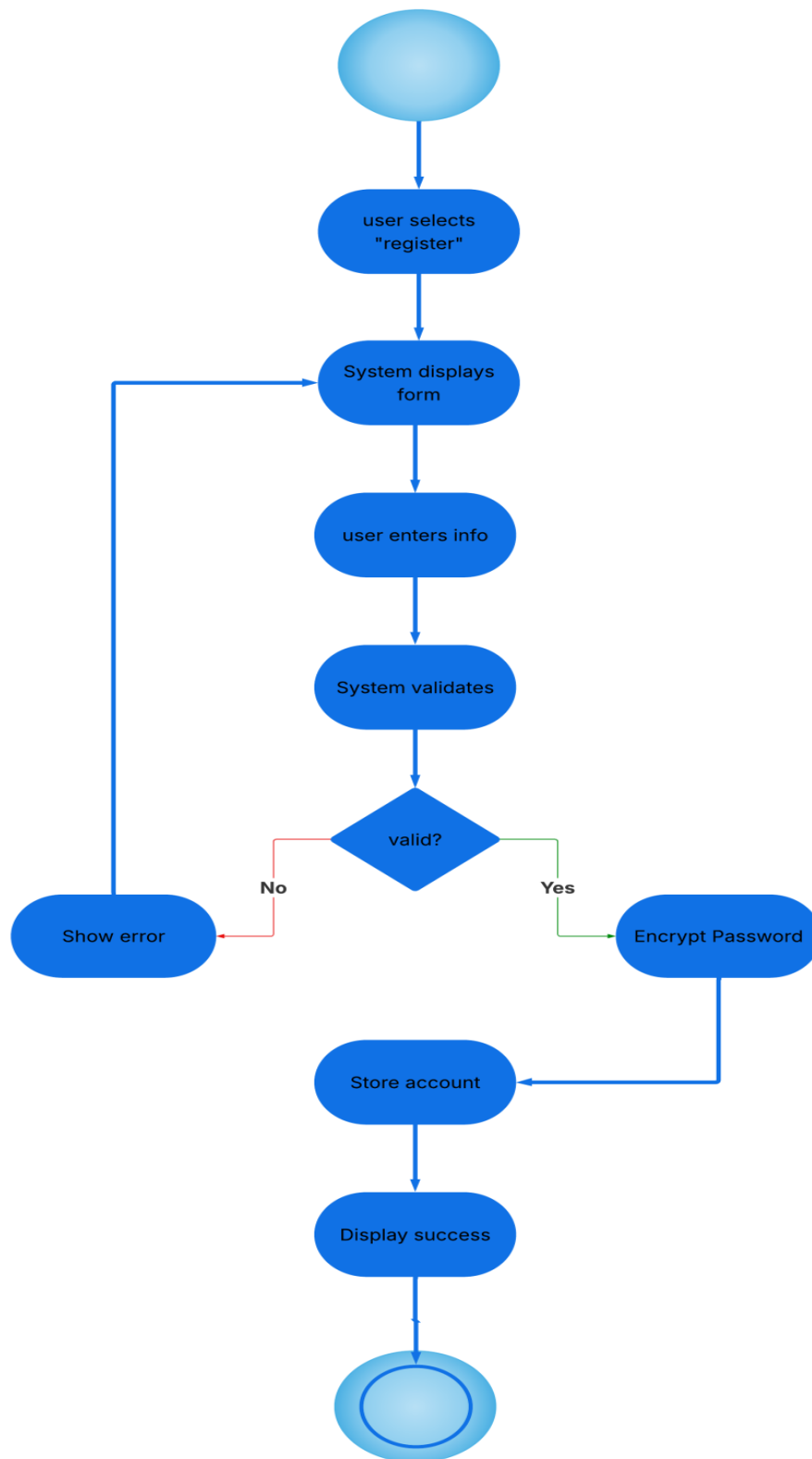


Figure 3.5: Activity Diagram - Register

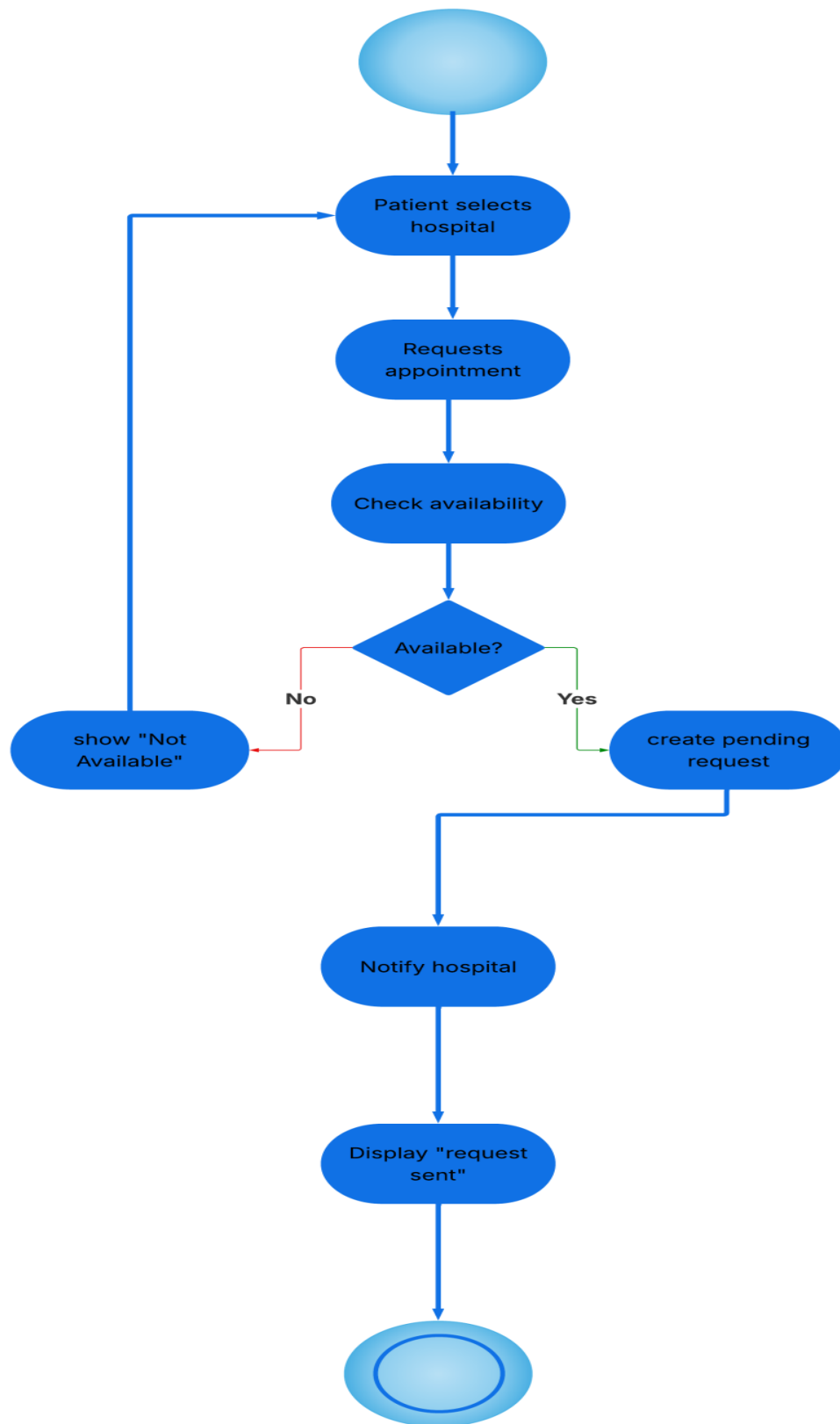


Figure 3.6: Activity Diagram – Request Appointment

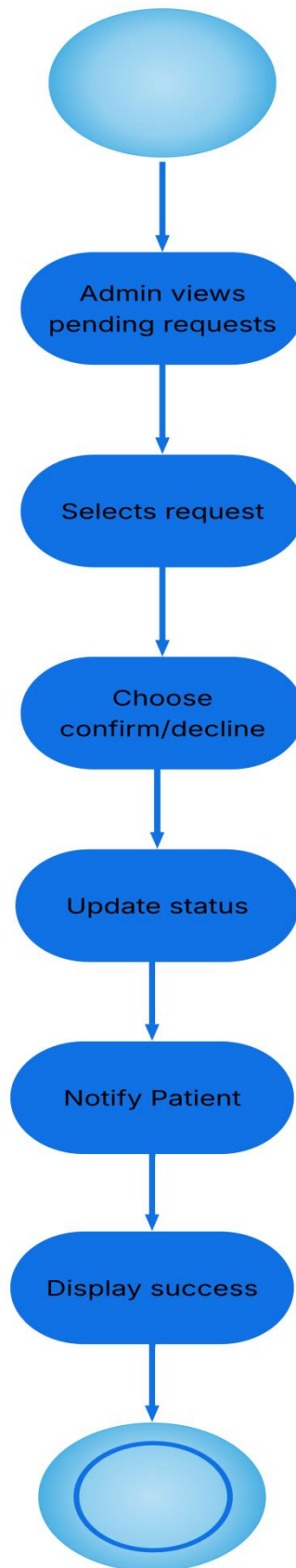


Figure 3.7: Activity Diagram – Confirm/Decline Appointment

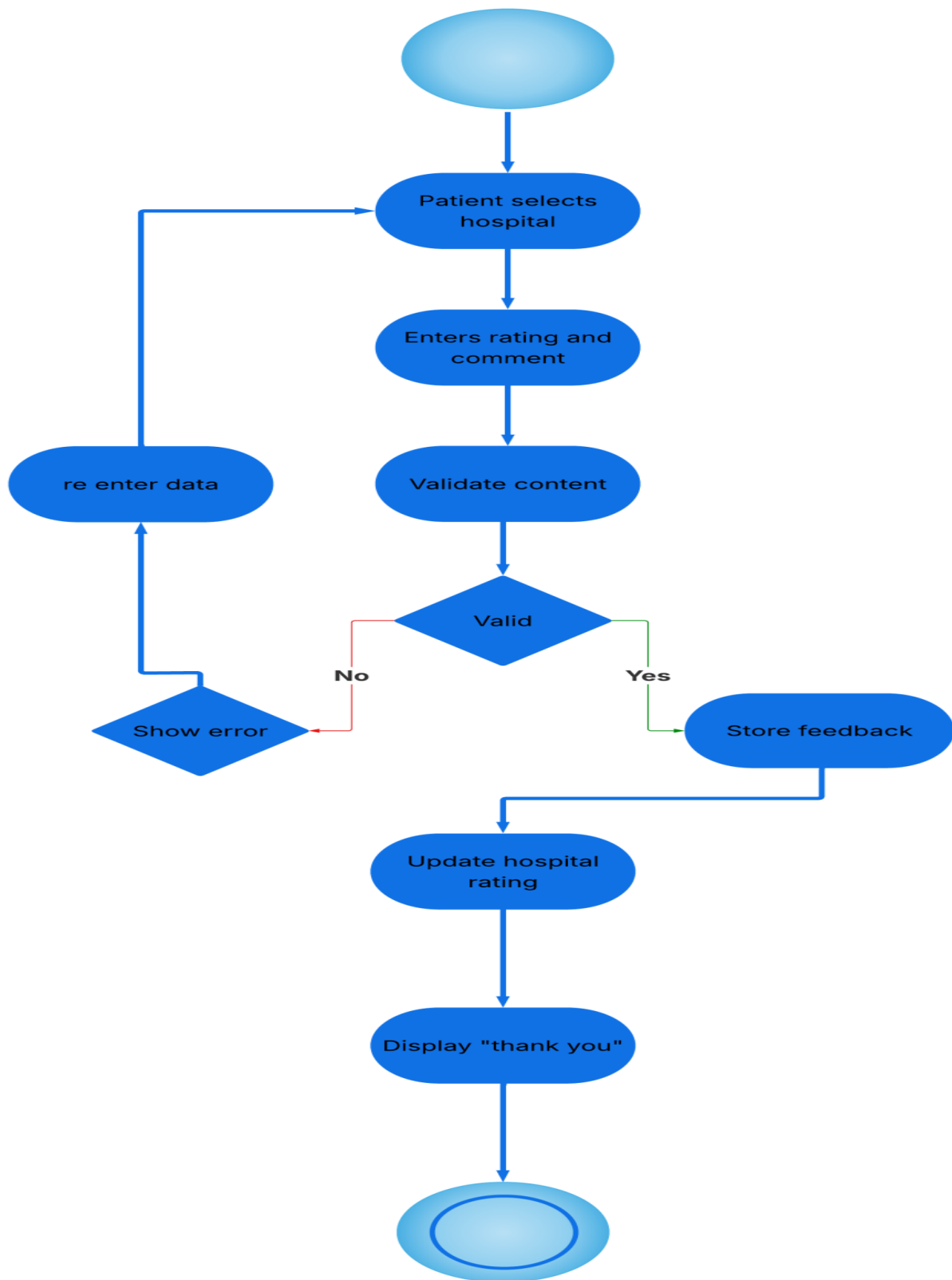


Figure 3.8: Activity Diagram – Submit Feedback

CHAPTER FOUR: SYSTEM DESIGN

4.1 Design Class Modeling

Design class modeling refines the analysis class diagram (Figure 3.3) by adding implementation-specific details such as method signatures, visibility, navigation, and persistence considerations. The design class model reflects technology decisions and serves as a blueprint for coding.

4.1.1 Design Class Diagrams

The design class diagram for CareConnect is shown in Figure 4.1. It includes the main classes from the analysis model (Patient, Hospital, Appointment, Review, MatchingEngine, User, HospitalAdmin, SystemAdmin, NotificationService) with the following enhancements:

- Visibility: Public (+), protected (#), private (-) are indicated for all attributes and methods.
- Method signatures: Parameters, types, and return types are specified.
- Navigation: Unidirectional associations are modeled where appropriate (e.g., from Appointment to Patient, not the reverse).
- Persistence: Classes that require persistent storage are stereotyped <<entity>>.
- Controller classes: MatchingEngine and NotificationService are stereotyped <<control>>.

The diagram also shows layering: user interface classes (SearchUI, AppointmentUI, AdminUI) in the UI layer, business domain classes in the

domain layer, and persistence classes (e.g., PersistenceManager) in the persistence layer.

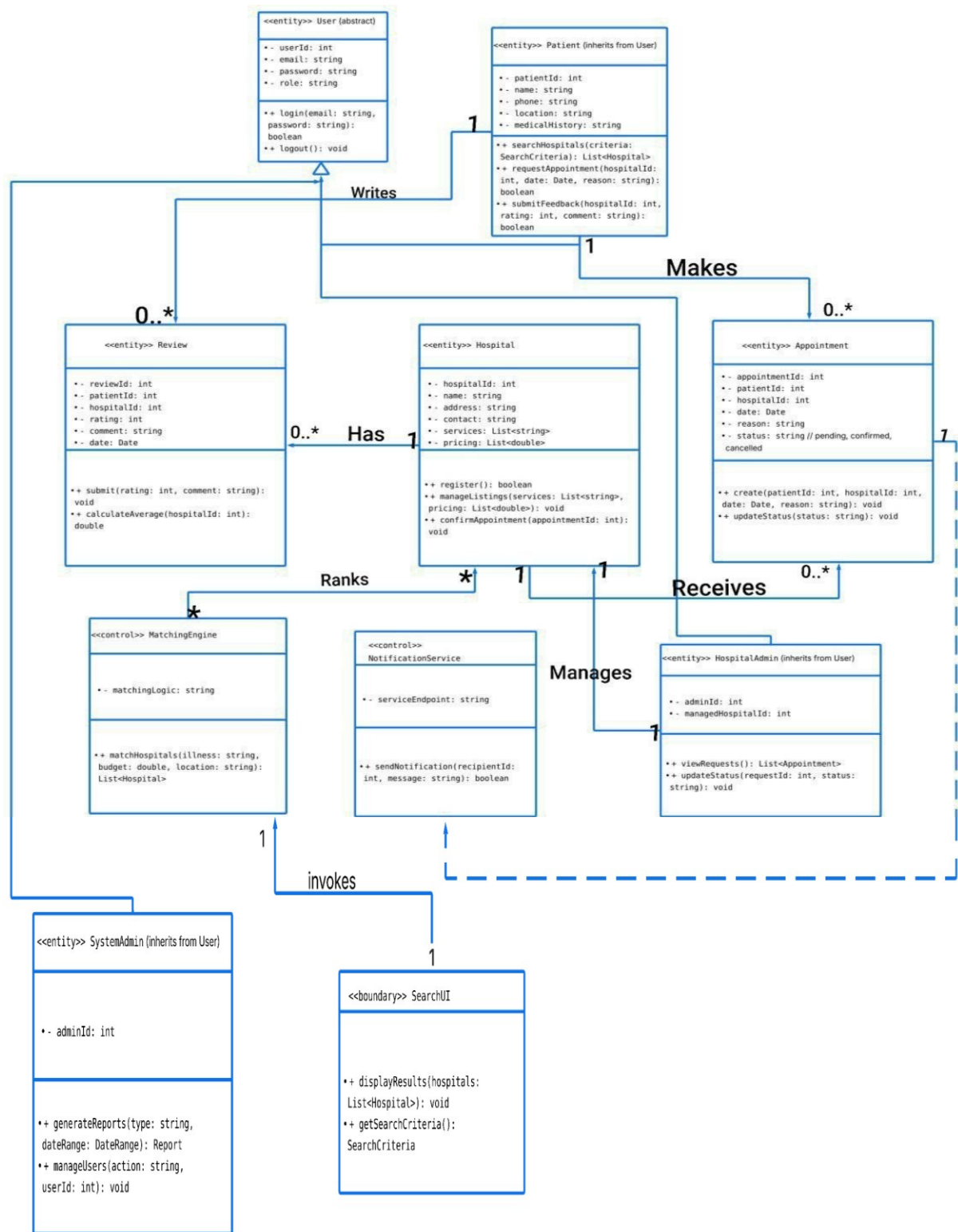


Figure 4.1: Design Class Diagram for CareConnect

4.1.2 Design Class Documentation

The following table documents each design class, its layer, key methods, and visibility.

Class	Layer	Key Methods (with signatures)	Visibility
Patient	Domain	+ searchHospitals(criteria: String): List<Hospital> + requestAppointment(hospitalId: int, date: Date): boolean + submitFeedback(hospitalId: int, rating: int, comment: String): boolean	Public
Hospital	Domain	+ register(): boolean + manageListings(services: List<String>, prices: List<Double>): void + confirmAppointment(appointmentId: int): void	Public
Appointment	Domain	+ create(patientId: int, hospitalId: int, date: Date, reason: String): void + updateStatus(status: String): void	Public
Review	Domain	+ submit(rating: int, comment: String): void + calculateAverage(hospitalId: int): double	Public
MatchingEngine	Control	+ matchHospitals(illness: String, budget: double, location: String): List<Hospital>	Public
NotificationService	Control	+ sendNotification(recipientId: int, message: String): Boolean	Public
User	Domain	+ login(email: String, password: String):	Public

(abstract)		Boolean + logout(): void	
HospitalAdmin	Domain	+ viewRequests(): List<Appointment> + updateStatus(requestId: int, status: String): void	Public
SystemAdmin	Domain	+ generateReports(type: String, dateRange: DateRange): Report + manageUsers(action: String, userId: int): void	Public
SearchUI	UI	+ displayResults(hospitals: List<Hospital>): void + getSearchCriteria(): SearchCriteria	Public
AppointmentUI	UI	+ showForm(): void + confirmSubmission(): void	Public
AdminUI	UI	+ showPendingRequests(): void + showSuccess(message: String): void	Public

Table 4.1 – Design Class Documentation

All attributes are private (-). Getters/setters are omitted for brevity.

4.2 Collaboration Diagrams

A collaboration diagram emphasizes the structural organization of objects that send and receive messages. It is an alternative view to a sequence diagram and is particularly helpful for visualizing how objects are linked.

For CareConnect, Figure 4.2 shows the collaboration diagram for the Search Hospitals use case. The diagram includes the following objects:

- :Patient (actor)
- :SearchUI (boundary)
- :MatchingEngine (control)
- :HospitalDatabase (entity)

The numbered messages indicate the order of interaction:

1. enter illness, budget, location – from :Patient to :SearchUI
2. validate input – self-call on :SearchUI
3. invokeMatch(illness, budget, location) – from :SearchUI to :MatchingEngine
4. queryHospitals() – from :MatchingEngine to :HospitalDatabase
5. return hospital list – from :HospitalDatabase to :MatchingEngine
6. rank hospitals – self-call on :MatchingEngine
7. return ranked list – from :MatchingEngine to :SearchUI
8. display results – from :SearchUI to :Patient

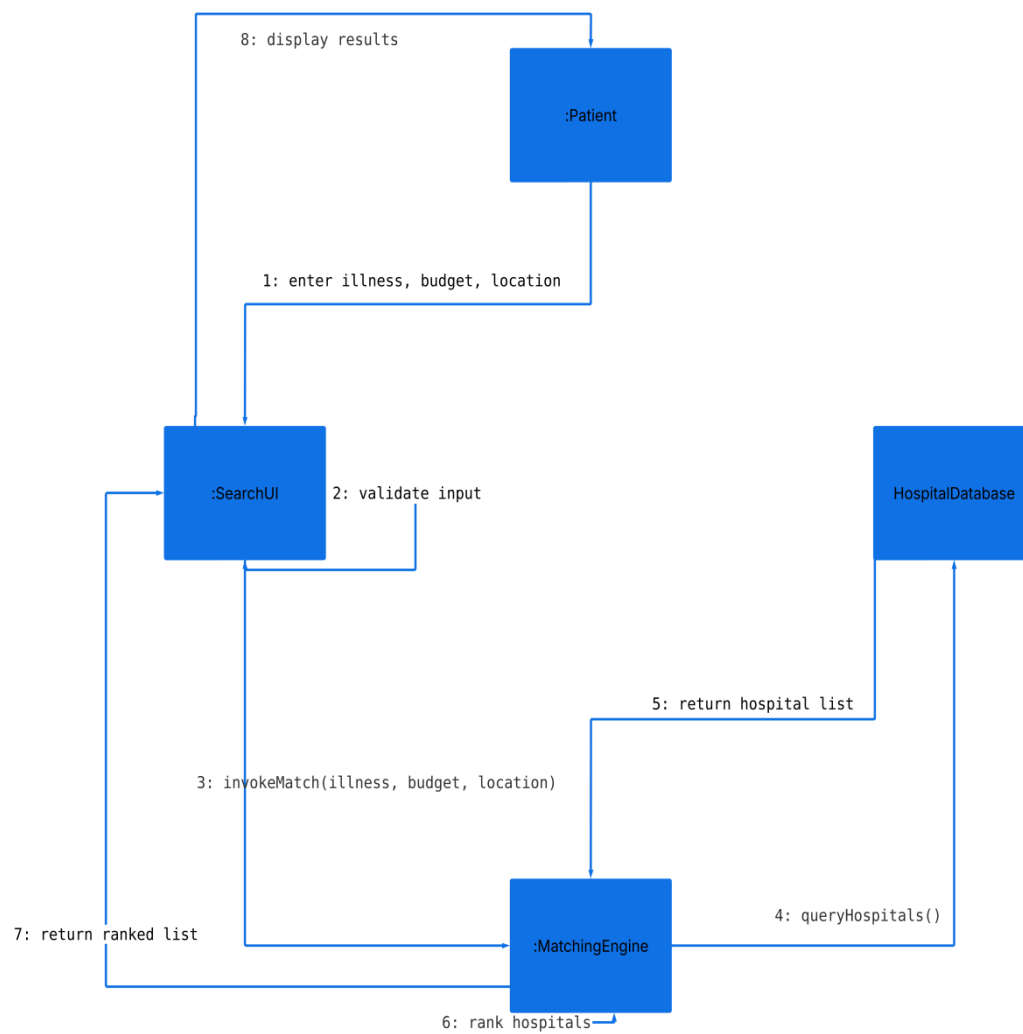


Figure 4.2: Collaboration Diagram – Search Hospitals

4.3 Component Diagrams

Component diagrams model the physical packaging and dependencies of software components. For CareConnect, Figure 4.3 shows the component architecture.

The main components and their stereotypes are:

- «web» CareConnectUI – the user interface (pages, forms, dashboards).
- «subsystem» BusinessLogic – contains controllers and domain classes that implement business rules.

- «subsystem» MatchingEngine – encapsulates the hospital ranking algorithm.
- «subsystem» Persistence – provides object relational mapping and data access.
- «external» NotificationService – a third party email/SMS service.
- «dataset» MatchingRules – stores illness to specialty mapping and proximity weights.
- «dataset» HospitalDataSet – stores raw hospital data (name, address, services, pricing).
- «database» HospitalDB – the relational database (MySQL/PostgreSQL).

Dependencies (with specific labels):

- CareConnectUI → BusinessLogic : invokes
- CareConnectUI → MatchingEngine : displays
- CareConnectUI → Persistence : requests data
- BusinessLogic → MatchingEngine : queries
- BusinessLogic → Persistence : stores/retrieves
- BusinessLogic → NotificationService : triggers
- MatchingEngine → MatchingRules : reads
- MatchingEngine → HospitalDataSet : loads
- Persistence → HospitalDB : maps

These dependencies ensure a modular architecture where each component has a clear, single responsibility. The matching engine is isolated from the database; it only reads datasets. The business logic coordinates actions but does not directly access the database or the notification service.

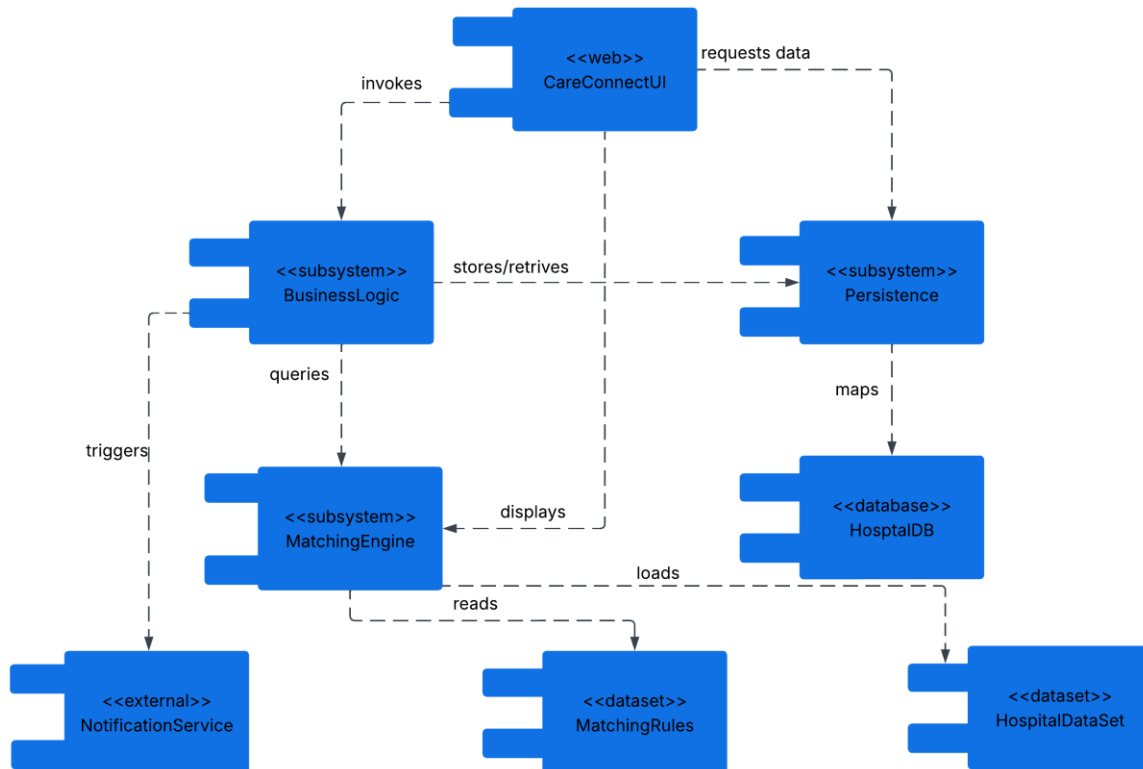


Figure 4.3: Component Diagram for CareConnect

4.4 Deployment Diagram

The deployment diagram (Figure 4.4) illustrates the physical runtime environment for CareConnect. The system is deployed as a web application with the following nodes:

- «device» ClientBrowser – any modern web browser (Chrome, Firefox) on a user’s computer or mobile device.
- «server» WebServer – hosts the CareConnectWeb component and the MatchingEngine component.
- «server» DatabaseServer – hosts the relational database (MySQL/PostgreSQL).

Artifacts (e.g., careconnect.war) are deployed on the WebServer. Communication between nodes uses HTTP/HTTPS (between ClientBrowser and WebServer) and JDBC (between WebServer and DatabaseServer).

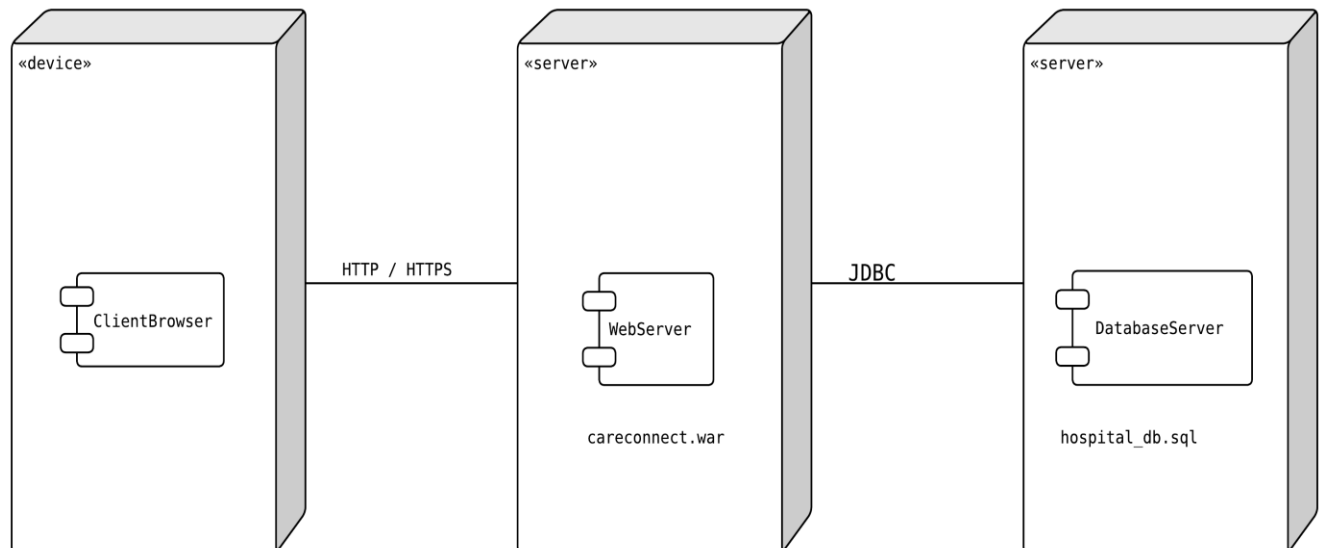


Figure 4.4: Deployment Diagram for CareConnect

4.5 Database Design

The database design translates the domain classes into a relational schema that ensures data integrity, efficiency, and scalability. CareConnect uses a relational database (MySQL/PostgreSQL) with the following design decisions.

4.5.1 Persistence Modeling

Persistence modeling maps the object-oriented domain model to a relational database schema. For CareConnect, the following tables are derived from the design class diagram (Figure 4.1):

- User – stores login credentials and role (Patient, HospitalAdmin, SystemAdmin).
- Patient – extends User, stores patient-specific data (name, phone, location, medical history).
- Hospital – stores hospital details (name, address, contact, services, pricing).
- HospitalAdmin – maps hospital administrators to the hospital they manage.
- Appointment – stores appointment requests (patientId, hospitalId, date, reason, status).
- Review – stores patient ratings and comments for hospitals.
- MatchingEngine – no persistent data; results are computed at runtime.

Primary keys: userId (User), patientId (Patient), hospitalId (Hospital), appointmentId (Appointment), reviewId (Review).

Foreign keys: patientId in Appointment references Patient; hospitalId in Appointment and Review references Hospital; managedHospitalId in HospitalAdmin references Hospital.

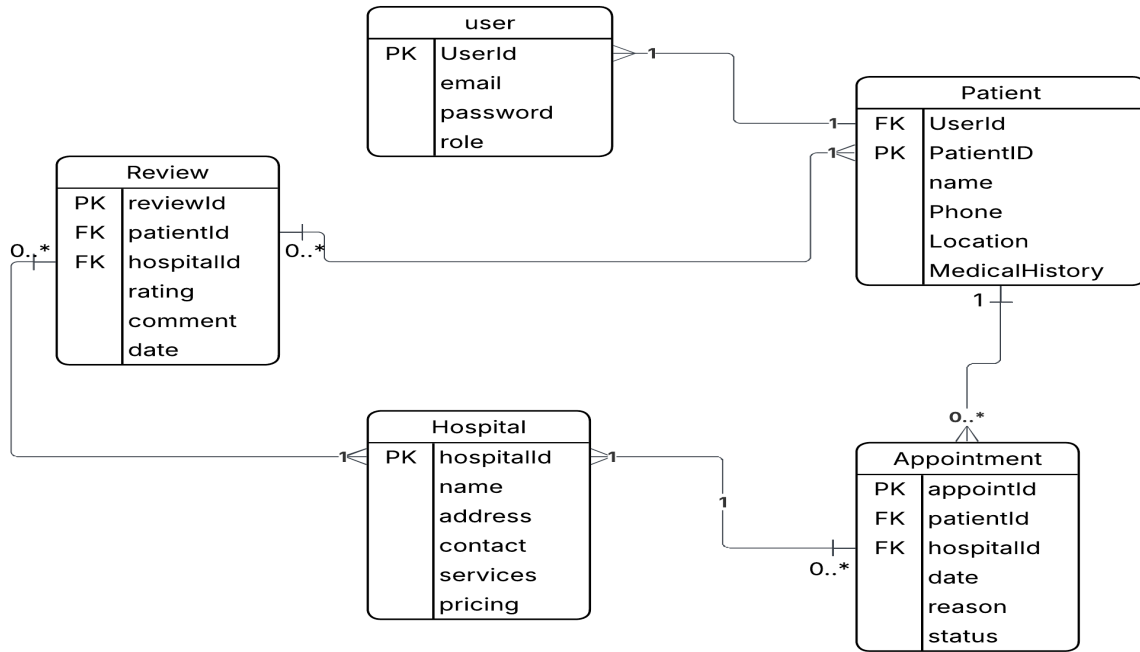


Figure 4.5: Persistence (ER) Diagram for CareConnect

4.5.2 Normalization

The database schema is normalized to the third normal form (3NF) to reduce redundancy and avoid update anomalies.

- First normal form (1NF) : All tables have atomic values and a primary key. For example, the Hospital table stores services and pricing as separate attributes; multi-valued fields are moved to separate tables if needed. In our design, services and pricing are stored as comma-separated strings or as a separate HospitalService table (for full normalisation, we would use a junction table, but given the simplicity, we keep them in the same table with appropriate validation).

- Second normal form (2NF) : All non-key attributes depend on the whole primary key. In Appointment, patientId and hospitalId determine the rest, no partial dependency exists.
- Third normal form (3NF) : No transitive dependencies. For instance, HospitalAdmin contains hospitalId directly, not through a surrogate.

The design is fully normalized for transactional operations, ensuring data integrity and consistency.

4.6 User Interface Design

User interface (UI) design focuses on the look-and-feel and interaction flow of the CareConnect system. The UI follows the principles of simplicity, visibility, feedback, and consistency.

UI Principles Applied:

- Structure: UI elements are grouped logically (search panel, results list, appointment form).
- Simplicity: Only necessary fields are shown; advanced filters are optional.
- Visibility: Important actions (search, request appointment, submit feedback) are clearly labeled and placed.
- Feedback: The system displays success/error messages after each user action (e.g., “Request sent”, “Invalid input”).

- Tolerance: Input validation prevents mistakes; users can correct errors without losing data.

Key UI Screens (Wireframes):

- Login/Registration Screen – allows users to sign up or log in.
- Patient Dashboard – shows search bar, recent searches, and appointment history.
- Search Results Screen – displays ranked list of hospitals with details (specialty, price, distance).
- Appointment Request Screen – form to select date, time, and reason.
- Hospital Admin Dashboard – shows pending appointment requests and listing management.
- System Admin Dashboard – report generation interface.

UI Flow Diagram (Figure 4.6) shows the navigation paths between screens for a typical patient session:

Login → Search → Select Hospital → Request Appointment → Logout.

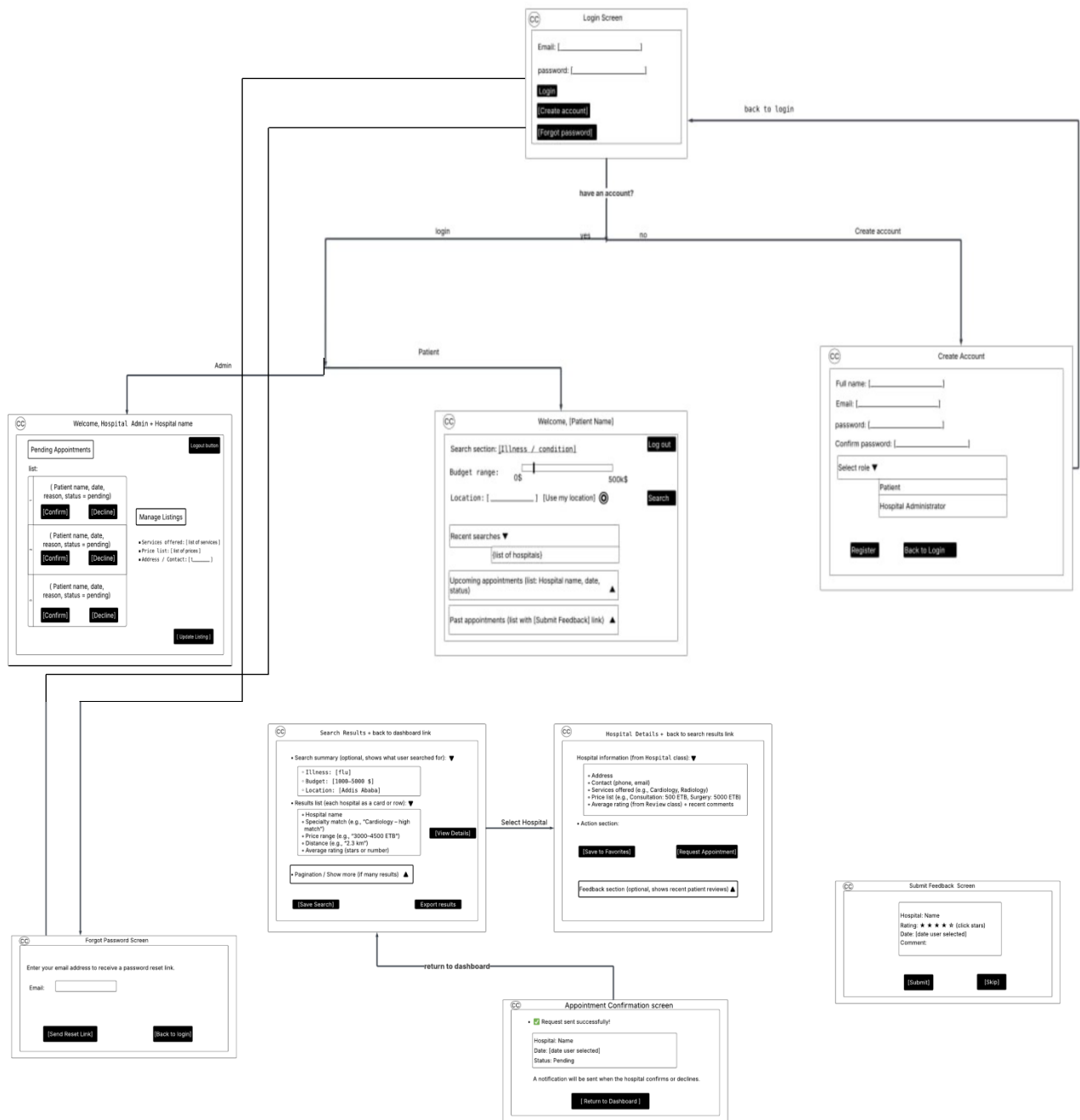


Figure 4.6: User Interface Flow Diagram for CareConnect

UI Prototype Notes

The final UI will be implemented as a responsive web application using HTML, CSS, and JavaScript. The prototype developed during analysis (Chapter 3) has been refined with input from potential users, simplifying the search form and adding real-time validation.

REFERENCE

- [1] Booch, G., Rumbaugh, J., and Jacobson, I. (2005) The Unified Modeling Language User Guide, 2nd ed., Addison-Wesley.
- [2] Pressman, R.S. (2014) Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill.
- [3] Unity University (2026) FINAL PROJECT WORK GUIDE LINE, Department of Computer Science.
- [4] Lucidchart (2026) Online Diagram Software, available at <https://www.lucidchart.com> (accessed 25 April 2026).